



APPLIED DEEP LEARNING

01 – TOOLS AND FRAMEWORKS

PROF. DR. LARS ACKERMANN-IGL

WHY DO WE START TALKING ABOUT TOOLS THOUGH WE DON'T KNOW THE PROBLEM, YET?



If we want to learn rather hands-on, we need to learn to speak the right language fast!

TOOLS AND FRAMEWORKS USED IN THIS COURSE



Plain old Python ☺

matplotlib



seaborn



TensorFlow



PyTorch



TOOLS AND FRAMEWORKS USED IN THIS COURSE

Development Environment



Plain old Python 😊



Visualization

matplotlib



seaborn

Data Preparation

pandas



NumPy

Development Environment



TensorFlow



PyTorch





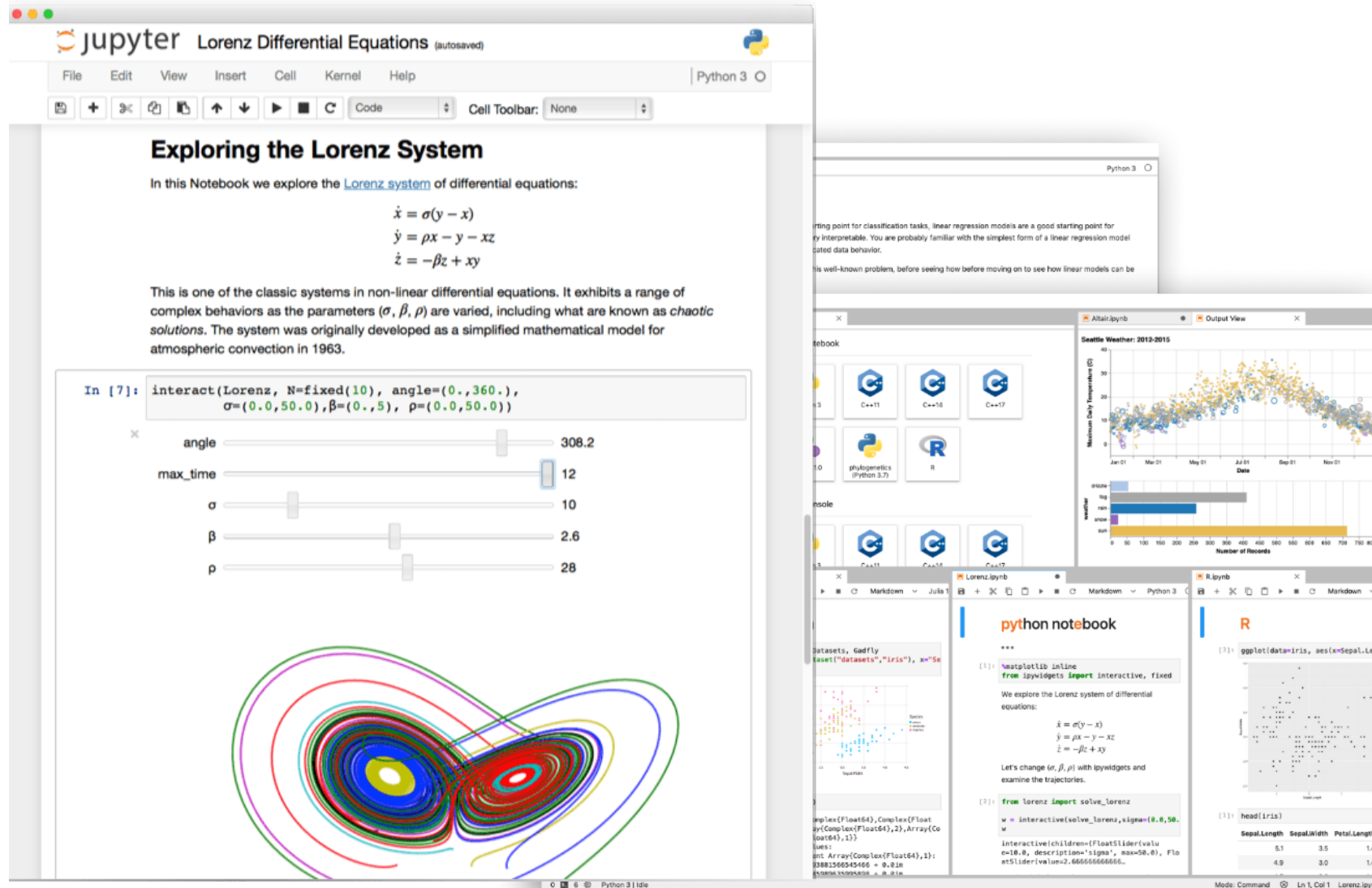
DEVELOPMENT ENVIRONMENT

- Jupyter
- Python projects with (Ana)conda support

Jupyter NOTEBOOKS



<https://jupyter.org/>



- Integrated code development (**code cells**) and documentation (**markdown cells**)
- Jupyter Notebook is a sequence of markdown and code cells
- **web-based** deployment
- developed for **fast** and **interactive experimentation**

Jupyter HUB @ IISYS



iisys

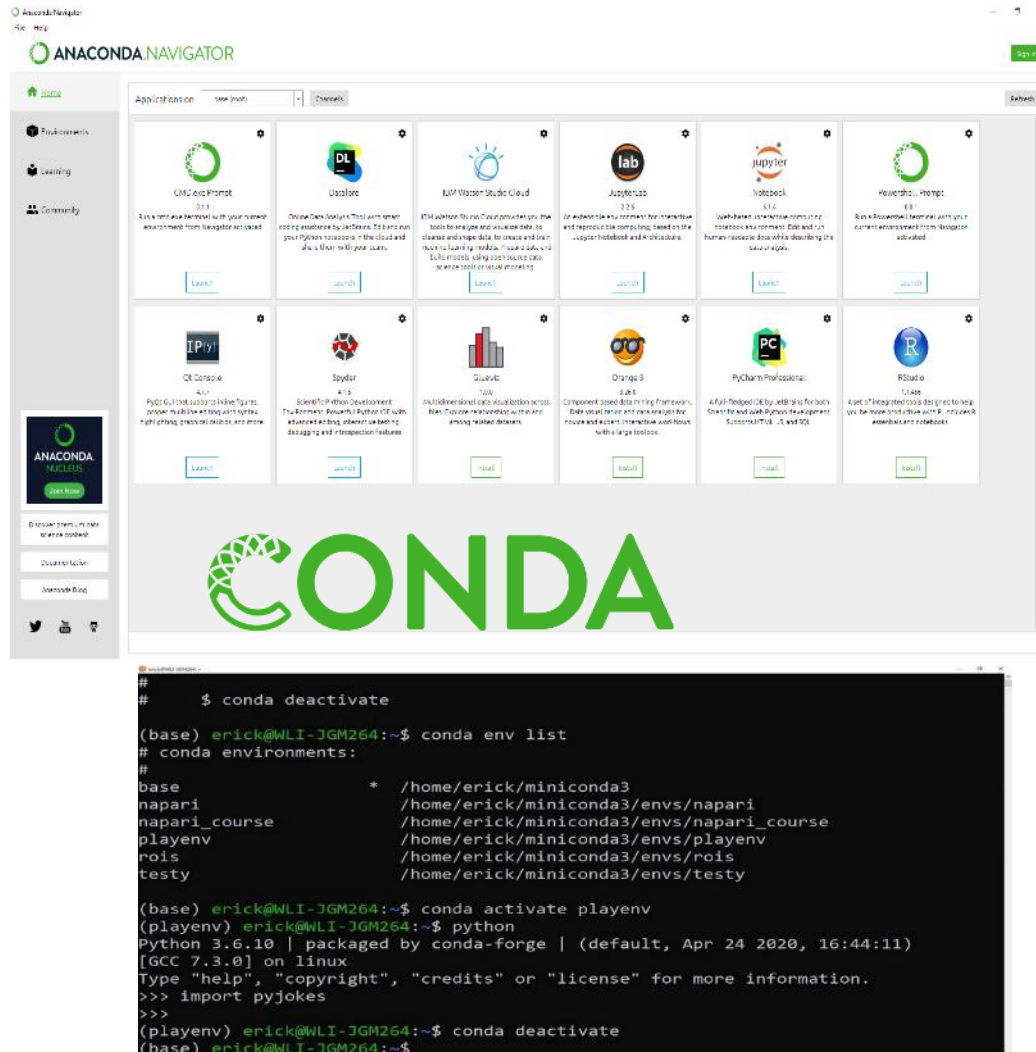
Institute for
Information Systems
at Hof University

<https://www.iisys.de/en/>

Name	Resource Key	Node Selector	Memory
apophis			
Name	Resource Key	Node Selector	Memory
NVIDIA GeForce RTX 4090	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-GeForce-RTX-4090	24 GB
kube-bastet			
Name	Resource Key	Node Selector	Memory
NVIDIA RTX 6800A/8000	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-RTX-6000	48 GB
kube-horus			
Name	Resource Key	Node Selector	Memory
mpg-1p-10g	nvidia.com/gpu/1p-10g	nvidia.com/gpu	10 GB
kube-isis			
Name	Resource Key	Node Selector	Memory
NVIDIA RTX PRO 4000 Blackwell Max Q Workstation Edition	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-RTX-PRO-4000-Blackwell-Max-Q-Workstation-Edition	96 GB
kube-montu			
Name	Resource Key	Node Selector	Memory
mpg-1p-10g	nvidia.com/gpu/1p-10g	nvidia.com/gpu	10 GB
mpg-1p-20g	nvidia.com/gpu/1p-20g	nvidia.com/gpu	20 GB
mpg-1p-40g	nvidia.com/gpu/1p-40g	nvidia.com/gpu	40 GB
kube-nut			
Name	Resource Key	Node Selector	Memory
NVIDIA GeForce RTX 4070 SUPER	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-GeForce-RTX-4070-SUPER	12 GB
kube-osiris			
Name	Resource Key	Node Selector	Memory
mpg-1p-20g	nvidia.com/gpu/1p-20g	nvidia.com/gpu	20 GB
mpg-1p-40g	nvidia.com/gpu/1p-40g	nvidia.com/gpu	40 GB
kube-ptah			
Name	Resource Key	Node Selector	Memory
NVIDIA GeForce RTX 4070 SUPER	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-GeForce-RTX-4070-SUPER	12 GB
kube-tefnut			
Name	Resource Key	Node Selector	Memory
NVIDIA RTX 4000 Ada Generation	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-RTX-4000-Ada-Generation	20 GB
kube-tefnut-2			
Name	Resource Key	Node Selector	Memory
NVIDIA RTX 6800A/8000	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-RTX-6800-Ada-Generation	48 GB
sekhmet			
Name	Resource Key	Node Selector	Memory
NVIDIA RTX PRO 4000 Blackwell Max Q Workstation Edition	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-RTX-PRO-4000-Blackwell-Max-Q-Workstation-Edition	96 GB
thot			
Name	Resource Key	Node Selector	Memory
NVIDIA H100 80G	nvidia.com/gpu	nvidia.com/gpu/product NVIDIA-H100-80G	80 GB

- **Preinstalled** and maintained infrastructure hosted at our university
-> „Jupyter as a Service“
- (Powerful) hardware in our data center

ANACONDA



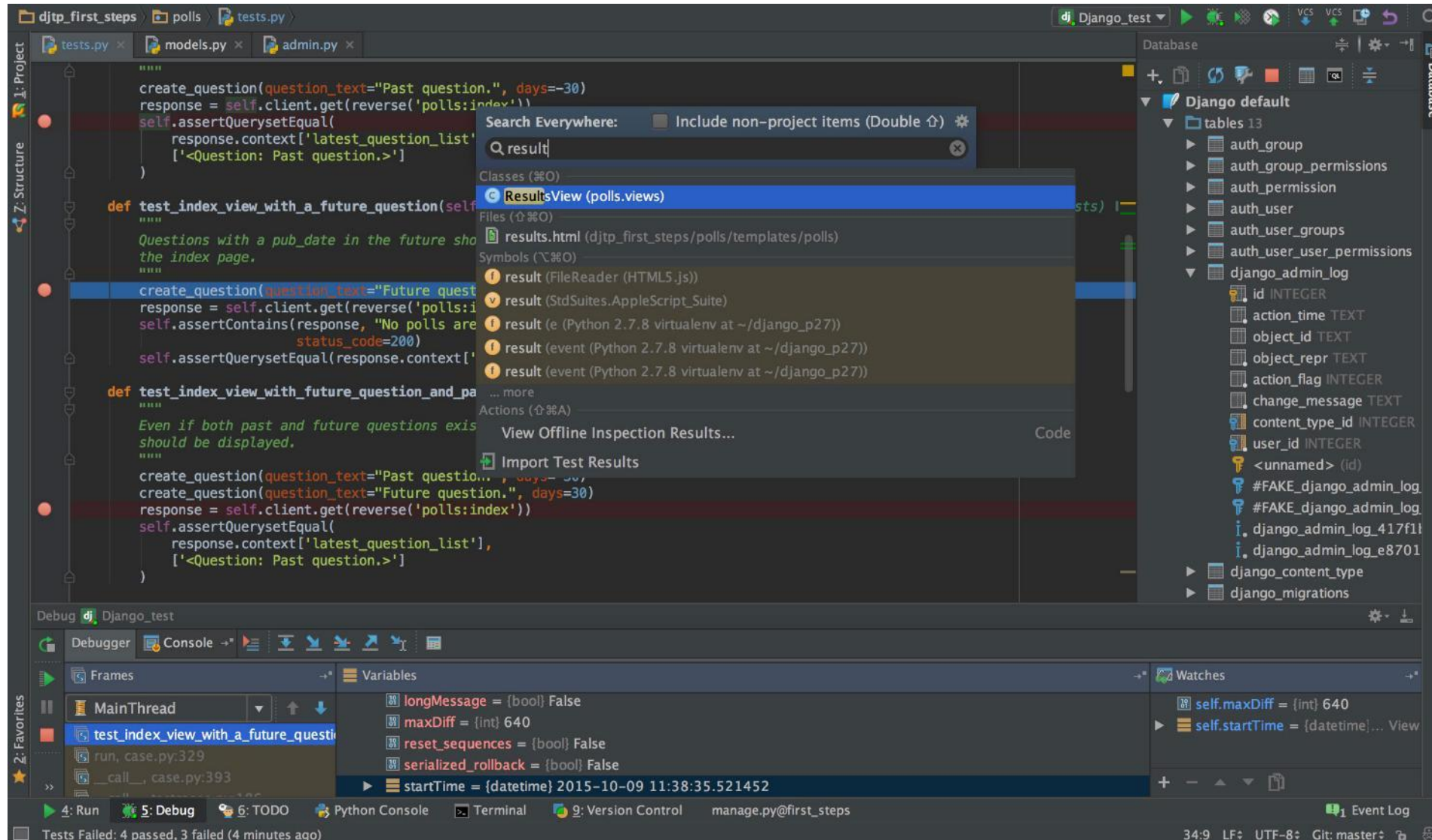
- **conda** is a command line tool for **package, dependency and environment management** for different programming languages
- Allows the definition of **different environments** (e.g. different python versions in combination with specific versions of libraries) and an **easy switch between the configurations**
- **Anaconda Navigator** is a **GUI** for conda
- Allows the **definition of application specific environments**
- Delivers **ready-to-go environments** for the relevant tools/frameworks **out-of-the-box**

PyCharm – POWERFUL IDE



<https://www.jetbrains.com/de-de/pycharm/>

- Developed by **JetBrains**, maybe you know them for **IntelliJ**
- Support of multiple programming languages
- Multiple plugins to common **version control systems** integrated
- (Very good) Alternative: VS Code





PYTHON (WITH PARALLELS TO JAVA)

- Our first Python program
- Data types
- Input, output and error messages
- Basic control flow
- Functions
- Classes, attribute visibility and inheritance
- Modules

WHY PYTHON?



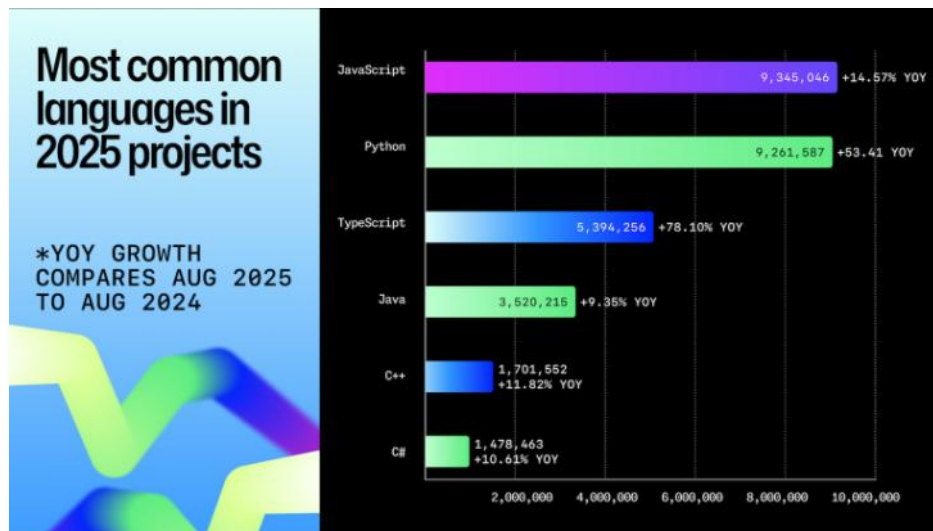
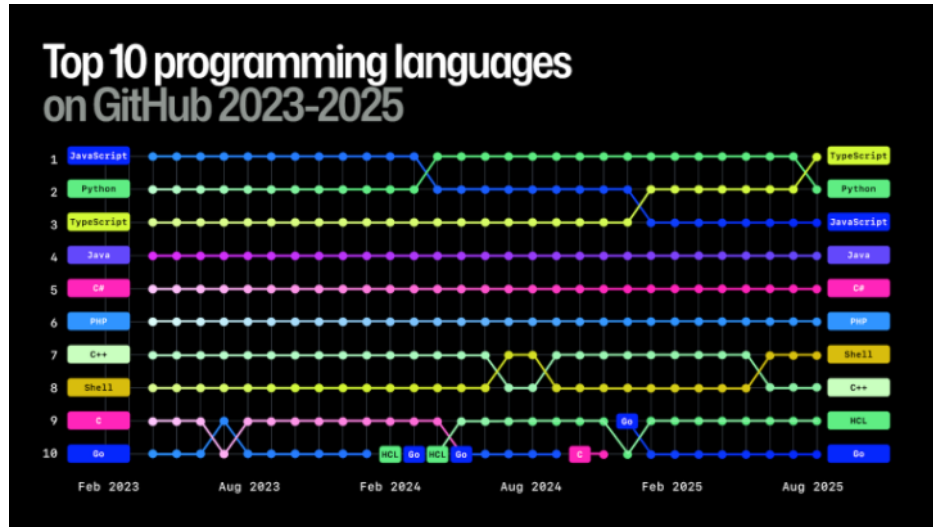
This year's data highlights three key shifts:

1. Generative AI is now standard in development. More than 1.1 million public repositories now use an LLM SDK with 693,867 of these projects created in just the past 12 months alone (+178% YoY, Aug '25 vs. Aug '24). Developers also merged a record 518.7M pull requests (+29% YoY). Moreover, AI adoption starts quickly: **80% of new developers on GitHub use Copilot in their first week.**

2. TypeScript is now the most used language on GitHub. In August 2025, TypeScript overtook both Python and JavaScript. Its rise illustrates how developers are shifting toward typed languages that make agent-assisted coding more reliable in production. It doesn't hurt that nearly every major frontend framework now scaffolds with TypeScript by default. Even still, **Python remains dominant for AI and data science workloads**, while the JavaScript/TypeScript ecosystem still accounts for more overall activity than Python alone.

3. AI is reshaping choices, not just code. In the past, developer choice meant picking an IDE, language, or framework. In 2025, that's changing. We see correlations between the rapid adoption of AI tools and evolving language preferences. This and other shifts suggest **AI is influencing not only how fast code is written, but which languages and tools developers use.**

And one of the biggest things in 2025? **Agents are here.** Early signals in our data are starting to show their impact, but ultimately point to one key thing: we're just getting started and we expect far greater activity in the months and years ahead.



https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/?utm_source=octoverse-homepage&utm_medium=blog&utm_campaign=universe25

OUR FIRST PYTHON PROGRAM

Java

```
List<Integer> numbers = Arrays.asList(1, -10, 0, -5, -1000, 100, 7);  
  
int maximum = numbers.get(0);  
  
for (int number : numbers) {  
    if (number > maximum) {  
        maximum = number;  
    }  
}  
  
System.out.println("The maximum value is " + maximum);
```

Python

```
numbers = [1, -10, 0, -5, -1000, 100, 7]  
  
maximum = numbers[0]  
  
for number in numbers:  
    if number > maximum:  
        maximum = number  
  
print("The maximum value is", maximum)
```

What are the similarities / differences between the Java and Python version?

What does the snippet do?



Try it yourself!

<https://www.online-python.com/>

OUR FIRST PYTHON PROGRAM

Java

```
List<Integer> numbers = Arrays.asList(1, -10, 0, -5, -1000, 100, 7);

int maximum = numbers.get(0);

for (int number : numbers) {
    if (number > maximum) {
        maximum = number;
    }
}

System.out.println("The maximum value is " + maximum);
```

Python

```
numbers = [1, -10, 0, -5, -1000, 100, 7]

maximum = numbers[0]

for number in numbers:
    if number > maximum:
        maximum = number

print("The maximum value is", maximum)
```

- There are no semicolons ;
- There are no types
- There are no braces { } but blocks that are started by a colon :
- There are no parentheses () around the expressions
- A list - one of many **built-in types** in Python - is initialized with square brackets []
- The for and if syntax is slightly different
- Comments are written with the hashtag symbol #

DATA TYPES

We will only look into a few non-container data types of Python, which Java would call **primitive types**.^{*}
Find all “built-in types” at docs.python.org/3/library/stdtypes.html

- Numeric Types
- Text Types
- Boolean Types

💡 In Python, you do not specify types explicitly! Although, you can have **type hints**.

* You can think of a variable in Python as a “tag” or “name” that is attached to some object, and NOT as the “container” that holds some value. All types in Python are essentially objects, even the numeric, text, and boolean ones. However, don’t wrap your head around this too much - as you will see, things turn out to behave quite similar to Java.

Python is dynamically-typed

Unlike Java, you can re-assign values of different types to the same variable as you wish.^{**}

```
x = 1416787301
x = "is"
x = True
```

** Speaking in the notion of “tags” and “names”, this means that you can freely re-assign the “tag” or “name” to point to a different location.

LET'S DO THE REMAINDER VIA JUPYTER

- Everything shown in the Jupyter notebook is also documented in the following and vice versa.
- The Jupyter notebook will not be shared via Moodle as we recommend to practice by yourself a bit. ;-)
- So, if you have your laptops/tablets with you, feel free to code with me.



powered by  **iisys**
Institut für
Informationssysteme
der Hochschule Hof

DATA TYPES – INDEXING / SEQUENCE SLICING

```
# ./python/m07_sequence_slicing.py#L1-L5

numbers = [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

numbers[5:8]    # give me a new COPY of the list, starting at index 5, and ending at index 7

# > [15, 16, 17]
```

The following variants are supported:

- `a[start:stop:step]`
- `a[start:stop]`
- `a[start:]`
- `a[:stop]`
- `a[:]`

The `stop` index is always exclusive, i.e. the element at index `stop` is not part of the resulting slice.

DATA TYPES – INDEXING / SEQUENCE SLICING

```
numbers = [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

```
# ./python/m07_sequence_slicing.py#L7-L14
```

```
numbers[1:]    # all except the 1st item    > [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]  
numbers[:3]    # the first three items    > [10, 11, 12]
```

```
numbers[:-1]   # all except the last item  > [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]  
numbers[-2:]   # the last two items       > [19, 20]
```

```
numbers[::2]    # every 2nd item           > [10, 12, 14, 16, 18, 20]  
numbers[1::2]   # every 2nd, but start at 2 > [11, 13, 15, 17, 19]
```

This also works with strings because they are just a list of characters.

```
# ./python/m07_sequence_slicing.py#L17-L20
```

```
name = "Hello World"  
name[1:-1]
```

```
# > 'ello Worl'
```

INPUT AND OUTPUT

You can read input from the command line with `input()`

```
name = input("Please enter your name: ")  
print("Your name is", name)
```

Don't forget data type conversions! Try it yourself: <https://www.online-python.com/>.

What happens if you run the code below?

```
number = input("Please enter a number: ")  
  
as_int = int(number)  
as_float = float(number)  
  
print(as_int, as_float)  
print("number+number: ", number*2)  
print("as_int+as_int: ", as_int + as_int)  
print("as_int+as_float: ", as_int + as_float)
```

ERROR MESSAGES

```
>>> print 17
File "<stdin>", line 1
  print 17
    ^
SyntaxError: Missing parentheses in call to 'print'
```

```
>>> print(seventeen)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'seventeen' is not defined
```

```
>>> print(11+'17')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Error messages in Python are (typically) quite helpful!

BASIC CONTROL FLOW

```
// ./java/M07_BasicControlFlow.java#L3-L17

int x = 0;

switch (x) {
    case 0:
        System.out.println("The value is 0");
        break;
    case 1:
        System.out.println("The value is 1");
        break;
    case 2:
        System.out.println("The value is 2");
        break;
    default:
        System.out.println("The value is something else");
}
```

```
# ./python/m07_basic_control_flow.py

x = 0

if x == 0:
    print("The value is 0")
elif x == 1:
    print("The value is 1")
elif x == 2:
    print("The value is 2")
else:
    print("The value is something else")
```

What do we notice here?

- Python has no `switch` expression, you have to make a cascade of `if` / `elif` / `else` statements*

* Actually, if you are using **Python 3.10** or newer, there are the new `match` and `case` statements now.

FUNCTIONS

```
static double area(double a, double b) {  
    return a * b;  
}  
  
static boolean isSquare(double a, double b) {  
    return a == b;  
}  
  
public static void main(String[] args) {  
    System.out.println("area(1, 5) = " + area(1, 5));  
    System.out.println("area(1.5, 2.3) = " + area(1.5, 2.3));  
  
    System.out.println("isSquare(1, 5) = " + isSquare(1, 5));  
    System.out.println("isSquare(5, 5) = " + isSquare(5, 5));  
}
```

```
def area(a, b):  
    return a * b  
  
def is_square(a, b):  
    return a == b  
  
print("area(1, 5) =", area(1, 5))  
print("area(1.5, 2.3) =", area(1.5, 2.3))  
  
print("is_square(1, 5) =", is_square(1, 5))  
print("is_square(5, 5) =", is_square(5, 5))
```

What do we notice here?

- Functions are defined with `def`
- Python does not have any access modifiers nor the `static` keyword
- You don't have to specify any types on the function parameters - this is called **duck-typing**

FUNCTIONS

You can specify argument names explicitly - and even change their order.

```
# ./python/m08_rectangle_functions.py#L17-L18  
  
area(a=2, b=5)  
area(b=5, a=2)
```

You can specify default values for arguments.

```
# ./python/m08_function_differences.py#L1-L12  
  
def function_with_default_args(x, y, name="Unknown User", factor=1.2):  
    result = x * y * factor  
    print(f"Hello {name}, your result is {result}")  
  
function_with_default_args(1, 2)  
function_with_default_args(1, 2, name="Mario")  
function_with_default_args(1, 2, factor=10)  
  
# > Hello Unknown User, your result is 2.4  
# > Hello Mario, your result is 2.4  
# > Hello Unknown User, your result is 20
```

FUNCTIONS

You can return multiple results with tuples and destructuring.

```
# ./python/m08_function_differences.py#L15-L30
```

```
def function_with_two_return_values(radius):  
    pi = 3.14159  
  
    circumference = 2 * pi * radius  
    area = pi * radius ** 2  
  
    return circumference, area
```

```
c, a = function_with_two_return_values(5)
```

```
print("circumference", c)  
print("area", a)
```

```
# > circumference 31.4159  
# > area 78.53975
```

CLASSES

Python is object-oriented, but follows a more lean approach towards classes.

```
// ./java/M09_PassengersProgram.java#L3-L40

class Passenger {

    final String firstName;
    final String lastName;

    Passenger(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    void display() {
        System.out.printf("%s %s %n", firstName, lastName);
    }

    static Passenger fromInput() {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter first name: ");
        String firstName = scanner.nextLine();
        System.out.print("Enter last name: ");
        String lastName = scanner.nextLine();

        scanner.close();

        return new Passenger(firstName, lastName);
    }
}
```

```
1 # ./python/m09_passengers_program.py
2
3 class Passenger:
4     def __init__(self, first_name, last_name):
5         self.first_name = first_name
6         self.last_name = last_name
7
8     def display(self):
9         print(f"Passenger: {self.first_name} {self.last_name}")
10
11     @staticmethod
12     def from_input():
13         first_name = input("Enter first name: ")
14         last_name = input("Enter last name: ")
15
16         return Passenger(first_name, last_name)
17
18
19 if __name__ == "__main__":
20     lisa = Passenger("Lisa", "Ha")
21     user = Passenger.from_input()
22
23     lisa.display()
24     user.display()
```


CLASSES

```
}  
void display() {  
    System.out.printf("%s %s %n", firstName, lastName);  
}  
  
static Passenger fromInput() {  
    Scanner scanner = new Scanner(System.in);  
  
    System.out.print("Enter first name: ");  
    String firstName = scanner.nextLine();  
    System.out.print("Enter last name: ");  
    String lastName = scanner.nextLine();  
  
    scanner.close();  
  
    return new Passenger(firstName, lastName);  
}  
  
class PassengersProgram {  
    public static void main(String[] args) {  
        Passenger lisa = new Passenger("Lisa", "Ha");  
        Passenger user = Passenger.fromInput();  
  
        lisa.display();  
        user.display();  
    }  
}
```

```
1 # ./python/m09_passengers_program.py  
2  
3 class Passenger:  
4     def __init__(self, first_name, last_name):  
5         self.first_name = first_name  
6         self.last_name = last_name  
7  
8     def display(self):  
9         print(f"Passenger: {self.first_name} {self.last_name}")  
10  
11     @staticmethod  
12     def from_input():  
13         first_name = input("Enter first name: ")  
14         last_name = input("Enter last name: ")  
15  
16         return Passenger(first_name, last_name)  
17  
18  
19 if __name__ == "__main__":  
20     lisa = Passenger("Lisa", "Ha")  
21     user = Passenger.from_input()  
22  
23     lisa.display()  
24     user.display()
```

CLASSES – CLASS / INSTANCE VARIABLES

Generally speaking, instance variables are for data unique to each instance and class variables are for attributes and methods shared by all instances of the class:

```
class Dog:

    kind = 'canine'          # class variable shared by all instances

    def __init__(self, name):
        self.name = name    # instance variable unique to each instance

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.kind          # shared by all dogs
'canine'
>>> e.kind          # shared by all dogs
'canine'
>>> d.name          # unique to d
'Fido'
>>> e.name          # unique to e
'Buddy'
```

CLASSES – ATTRIBUTE VISIBILITY AND INHERITANCE

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.__update(iterable)

    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)

    __update = update    # private copy of original update() method

class MappingSubclass(Mapping):

    def update(self, keys, values):
        # provides new signature for update()
        # but does not break __init__()
        for item in zip(keys, values):
            self.items_list.append(item)
```

MODULES

```
import foo                # foo imported and bound locally
import foo.bar.baz        # foo, foo.bar, and foo.bar.baz imported, foo bound locally
import foo.bar.baz as fbb # foo, foo.bar, and foo.bar.baz imported, foo.bar.baz bound as fbb
from foo.bar import baz    # foo, foo.bar, and foo.bar.baz imported, foo.bar.baz bound as baz
from foo import attr       # foo imported and foo.attr bound as attr
```

The basic import statement (no [from](#) clause) is executed in two steps:

1. find a module, loading and initializing it if necessary
2. define a name or names in the local namespace for the scope where the [import](#) statement occurs.

The [from](#) form uses a slightly more complex process:

1. find the module specified in the [from](#) clause, loading and initializing it if necessary;
2. for each of the identifiers specified in the [import](#) clauses:
 1. check if the imported module has an attribute by that name
 2. if not, attempt to import a submodule with that name and then check the imported module again for that attribute
 3. if the attribute is not found, [ImportError](#) is raised.
 4. otherwise, a reference to that value is stored in the local namespace, using the name in the as clause if it is present, otherwise using the attribute name



MODELING

- Non-neural techniques: sklearn
- Neural techniques: TensorFlow or PyTorch

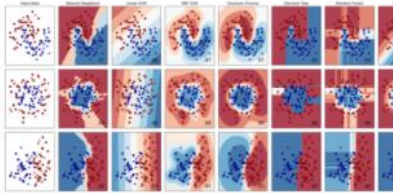
SKLEARN

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...



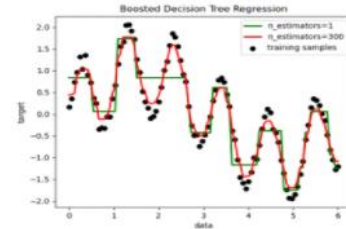
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...



Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...



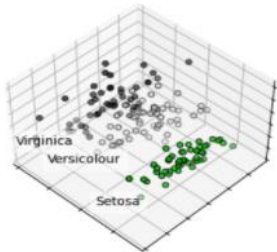
Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...



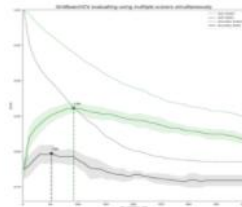
Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...



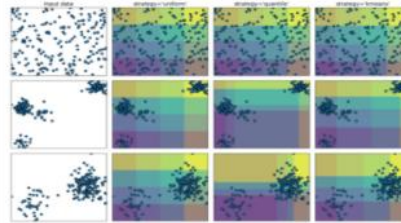
Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: preprocessing, feature extraction, and more...



Examples



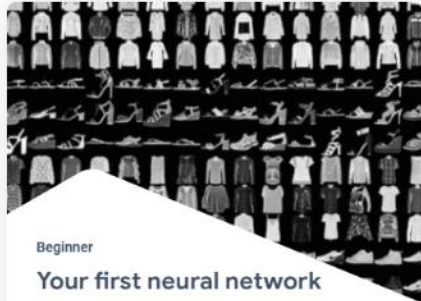
<https://scikit-learn.org/stable/>

- Provides **methods** for
 - Preprocessing
 - Classification
 - Regression
 - Clustering
 - Model selection and evaluation
- Some of the approaches we will focus in detail in our specific modelling lectures!

NEURAL TECHNIQUES – TENSORFLOW AND PYTORCH

Solutions to common ML problems

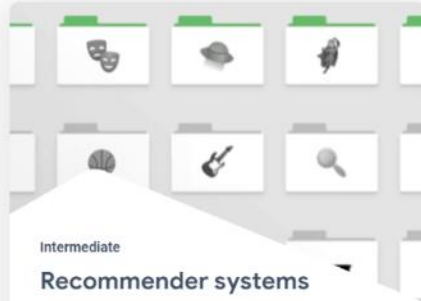
Simple step-by-step walkthroughs to solve common ML problems with TensorFlow.



Beginner

Your first neural network

Train a neural network to classify images of clothing, like sneakers and shirts, in this fast-paced overview of a complete TensorFlow program.



Intermediate

Recommender systems

Start with building and training a retrieval model to predict a set of movies that a user is likely to watch, and then use a ranking model to create recommendations.

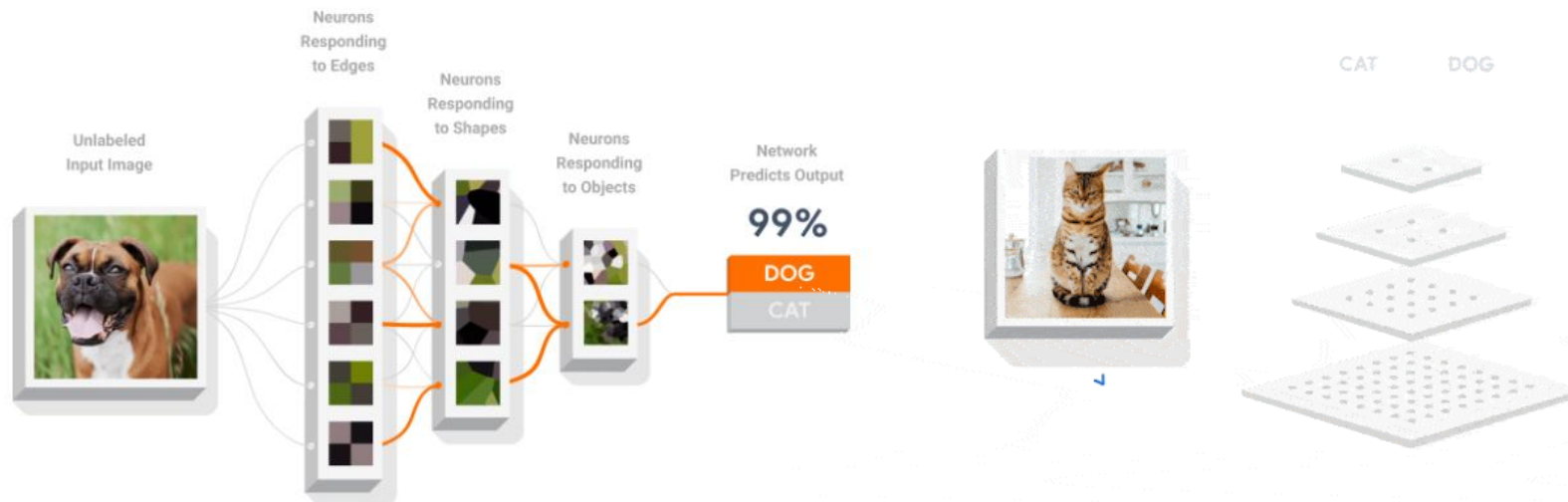


Expert

Generative adversarial networks

Train a generative adversarial network to generate images of handwritten digits, using the Keras Subclassing API.

...



- Tensorflow is a **low-level library** providing various models for data science problems
- Broad model zoo (especially focus on neural net based approaches)
- Often used in combination with Keras as high-level API
- Alternative: PyTorch
 - PyTorch: Better Python integration, rapid prototyping
 - Tensorflow: strong visualization capabilities, production-ready deployment options



DATA PREPARATION WITH NumPy & pandas

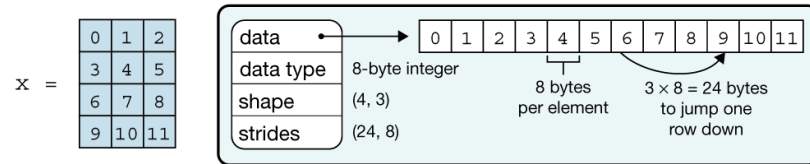
- NumPy: Efficient computing of numerical operations on arrays
- Pandas: Series for 1D data + DataFrames for 2D data

NumPy

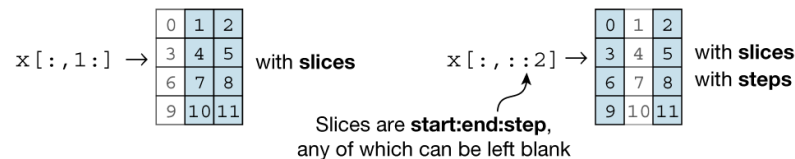


<https://numpy.org/>

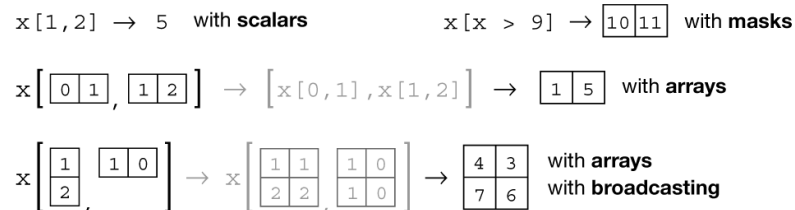
a Data structure



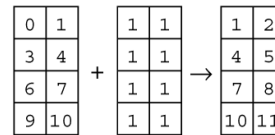
b Indexing (view)



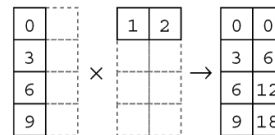
c Indexing (copy)



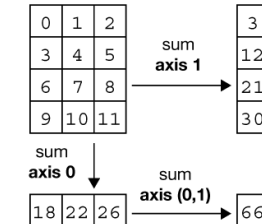
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
In [2]: x = np.arange(12)
In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[ -4.5,  -4.5,  -4.5],
       [ -1.5,  -1.5,  -1.5],
       [  1.5,   1.5,   1.5],
       [  4.5,   4.5,   4.5]])
```

- NumPy enables **fast, efficient numerical computing** with **multidimensional arrays**.
- First reason: implementation of many array operations in **C**
- Second (most important) reason: NumPy stores multi-dimensional data (tensors) at **one continuous place in memory**, enabling fast access and manipulation (locality of reference).
- NumPy forms the **basis for pandas** (next slide)

pandas

Series / DataFrame generation:

```
In [3]: s = pd.Series([1, 3, 5, np.nan, 6, 8])
```

```
In [4]: s
Out[4]:
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

```
In [5]: dates = pd.date_range("20130101", periods=6)
```

```
In [6]: dates
Out[6]:
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
              '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

```
In [7]: df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
```

```
In [8]: df
Out[8]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

```
In [229]: pd.read_json("test.json")
```

```
Out[229]:
```

	A	B	date	ints	bools
2013-01-01	-1.294524	0.413738	2013-01-01	0	True
2013-01-02	0.276662	-0.472035	2013-01-01	1	True
2013-01-03	-0.013960	-0.362543	2013-01-01	2	True
2013-01-04	-0.006154	-0.923061	2013-01-01	3	True
2013-01-05	0.895717	0.805244	2013-01-01	4	True

Missing-data handling:

```
In [42]: df2
```

```
Out[42]:
```

	one	two	three	four	five	timestamp
a	NaN	-0.282863	-1.509059	bar	True	NaT
c	NaN	1.212112	-0.173215	bar	False	NaT
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01
h	NaN	-0.706771	-1.039575	bar	True	NaT

```
In [43]: df2.fillna(0)
```

```
Out[43]:
```

	one	two	three	four	five	timestamp
a	0.000000	-0.282863	-1.509059	bar	True	0
c	0.000000	1.212112	-0.173215	bar	False	0
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01 00:00:00
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01 00:00:00
h	0.000000	-0.706771	-1.039575	bar	True	0

Describing DataFrames:

```
>>> s = pd.Series([1, 2, 3])
```

```
>>> s.describe()
```

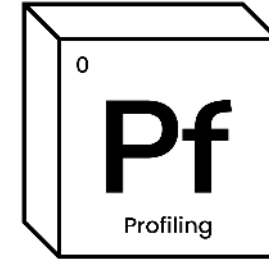
```
count    3.0
mean     2.0
std      1.0
min      1.0
25%     1.5
50%     2.0
75%     2.5
max      3.0
dtype: float64
```



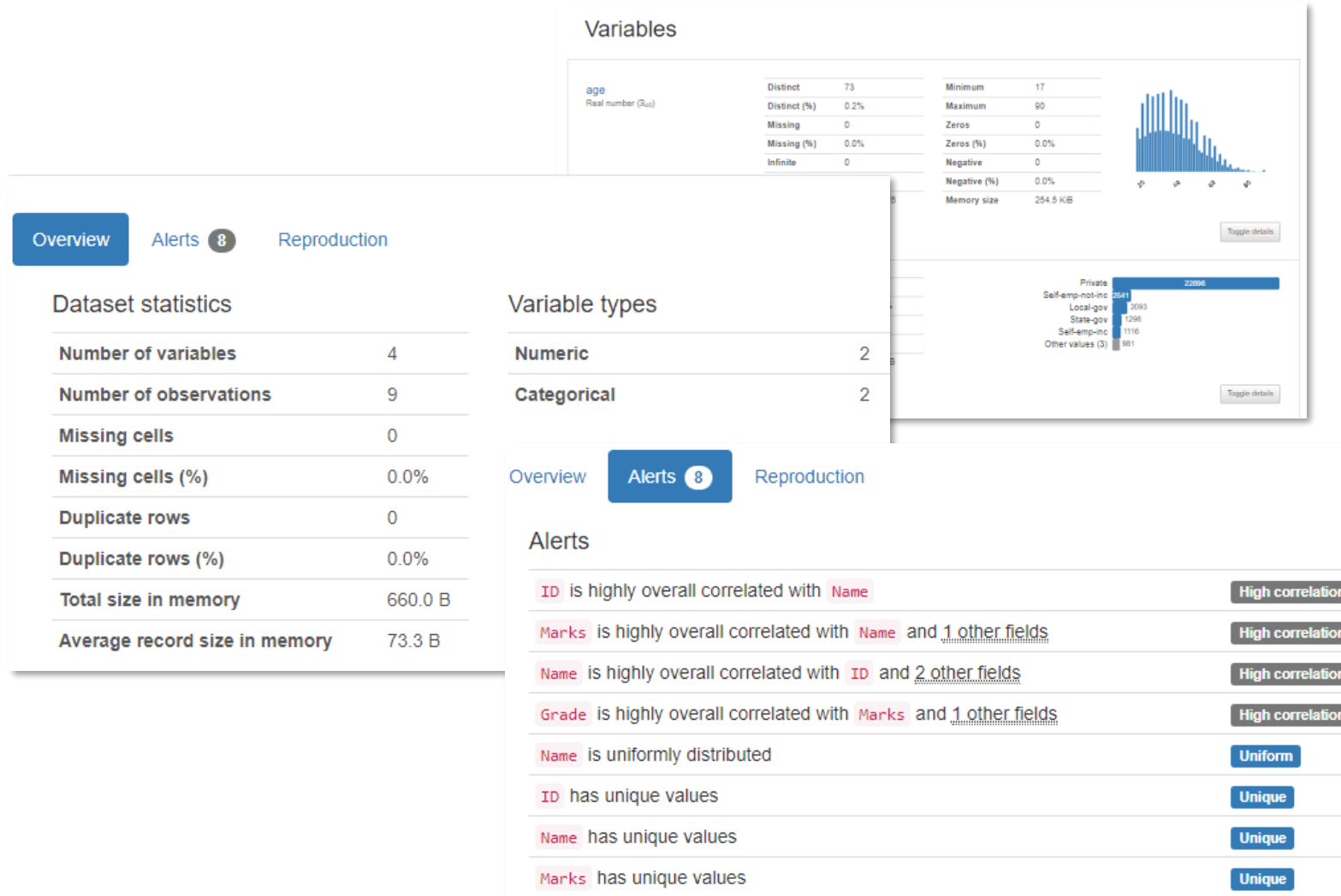
<https://pandas.pydata.org/>

- Library for data-relevant tasks like **data preprocessing and transformation**
- can be used with **numerical** and **categorical** data
- manipulation of 1D (Series) and 2D (DataFrame) structures
- Built-in features for indexing, cleaning, merging, selecting, ...
- Offers multiple connectors for different import/export types
- generation of statistics for a specific data frame (DataFrame.describe())
 - ydata-profiling (formerly: pandas-profiling)

ydata-profiling (pandas-profiling)



<https://github.com/ydataai/ydata-profiling>



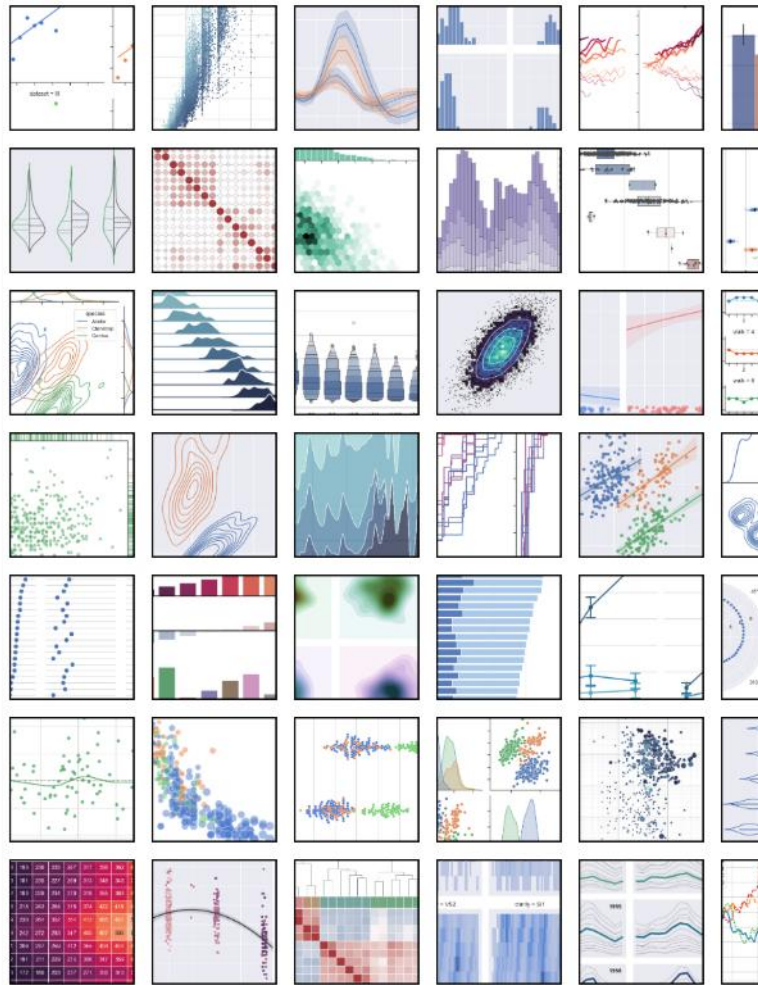
- Library for **one-line Exploratory Data Analysis (EDA)**
- **Extends pandas DataFrame.describe()** with detailed insights
- Supports numerical, categorical, time-series, and text data
- Generates **reports** in various formats (HTML, JSON, ...)
- Use it for a **concise and structured dataset analysis**



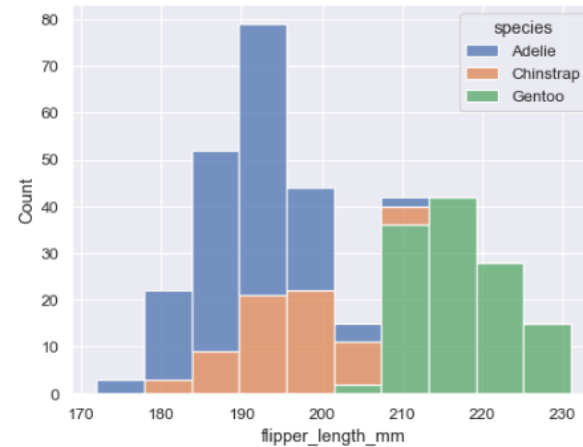
VISUALIZATION

- Plots
- Teaser: Choose the right chart type!

seaborn



```
penguins = sns.load_dataset("penguins")
sns.histplot(data=penguins, x="flipper_length_mm", hue="species", multiple="stack")
```



```
g = sns.relplot(data=penguins, x="flipper_length_mm", y="bill_length_mm", col="sex")
g.set_axis_labels("Flipper length (mm)", "Bill length (mm)")
```



matplotlib



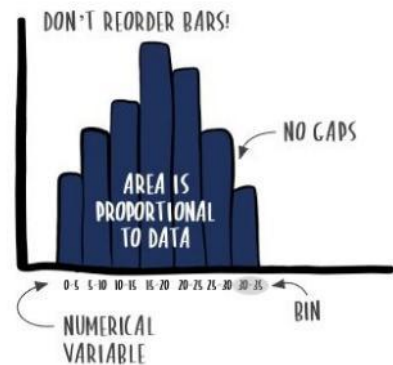
<https://seaborn.pydata.org/>

- Library for **visualization of data**
 - Distributions
 - Statistical relationships
 - Categorical data
 - Regression models
 - Multi-plot grids
- Static visualization of data
- Based **on matplotlib**
 - seaborn as high-level interface
- **Compatible with different data source types** like pandas dataframes

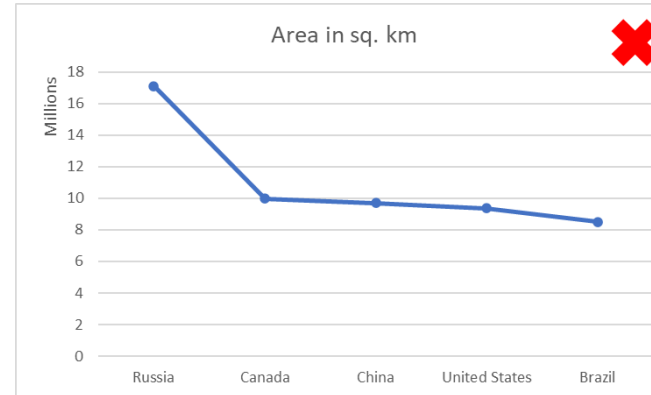
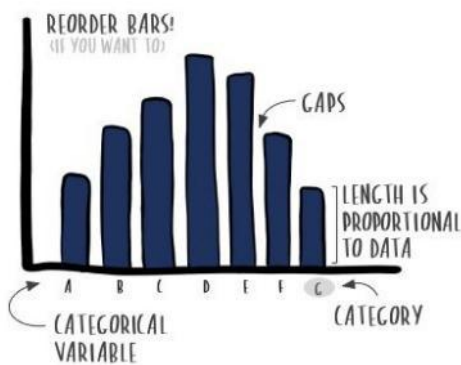
TEASER: CHOOSING THE RIGHT CHART TYPE

TWO TRICKY EXAMPLES

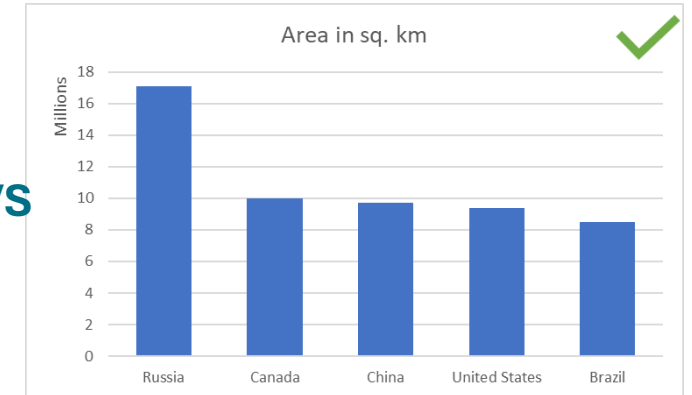
This is a histogram...



This is a bar chart...



VS



- Histograms visualize numerical (quantitative) data, while bar charts display categorical variables.
- Histograms visualize the **distribution** of a (typically continuous) variable by dividing the data into bins and counting how many observations fall into each bin.
- Bar charts represent a finite set of categories (e.g., boroughs in a city).
- Discrete numerical data can be shown in either a histogram or a bar chart, depending on the purpose and number of values.
- If discrete values vary widely, they are often treated as continuous and grouped into bins.

- Line charts emphasize continuity by connecting data points with a line.
- Avoid using line charts for discrete data, as it may falsely imply continuity.
- Discrete time values and categorical/nominal data should not be plotted with line charts.
- Example: Using a line chart for country areas incorrectly suggests a relationship between them.
- Better alternative: Use a bar chart for categorical comparisons.

QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de